NL-to-SQL Generator

Natural Language to SQL, with Execution & Self-Correction

Complete Build Guide — Architecture, Schema, Self-Correction Loop, Safety & Code

Project 2 of 3 — Resume Project Series

Stack: Python · FastAPI · LangChain · PostgreSQL · Redis · OpenAI · Docker

Prepared for Saksham Tiwari

KEY INSIGHT

This is NOT "call an LLM, show the answer." The LLM's SQL is a guess until it actually executes against a real schema. The core engineering here is: schema introspection so the model knows what exists, execution against a live database for real grounding, a self-correction loop that catches and fixes failed queries, and a read-only safety layer that stops a bad generation from ever mutating data.

1. What We're Building

A system that takes a plain-English question — "what were our top 5 products by revenue last quarter?" — and turns it into a real, executable SQL query, runs it against a live PostgreSQL database, and returns the actual result. Not a text guess. A real query, executed, with real rows back.

Why not just call an LLM directly?

An LLM with no context has no idea what tables or columns exist in your database. Asked directly, it either hallucinates plausible-sounding numbers, or correctly says it doesn't have access to your data. Neither is useful. The interesting engineering is entirely in closing that gap between "a language model that predicts text" and "a system that reliably produces safe, correct, executable queries against a schema it's never seen."

The five things that make this real engineering, not a wrapper

- **Schema introspection** — the system reads your actual Postgres schema (tables, columns, types, foreign keys) and feeds it into the prompt, so the model can only reference things that actually exist.
- **Execution as grounding** — the generated SQL is only a hypothesis until it runs. Executing it against the real database is the equivalent of RAG's retrieval step — it's what turns a guess into something grounded in reality.
- **Self-correction loop** — LLM-generated SQL fails constantly (wrong column, bad join, syntax error). The system catches the database error, feeds it back to the LLM, and retries with a hard cap — this is the core novel logic in the whole project.
- **Read-only safety enforcement** — a generated query must never be allowed to DROP, DELETE, UPDATE, or INSERT. This is enforced in code, not just prompted for.
- **Ambiguity handling** — "last month," "top," "recent" need explicit date logic and sort/limit handling baked into the prompt, not left to the model to guess consistently.

What's reused from Project 1 (DocuMind)

FastAPI patterns, the LangChain + OpenAI chain-building approach, lazy client initialization (learned the hard way in Project 1), and Docker Compose structure all carry over directly. What's new is almost entirely in Sections 4–6 below.

2. System Architecture

The self-correction loop is the box most tutorials skip entirely — and it's the part worth walking an interviewer through in detail (Section 5).

3. Tech Stack & Why Each Piece

Layer	Choice	Why
API framework	FastAPI	Same as Project 1 — async, auto docs, Pydantic validation
Orchestration	LangChain	Prompt templates + LLM invocation, same pattern as Project 1
Database	PostgreSQL	The actual data being queried — realistic target, not a toy
DB driver	asyncpg or psycopg2	asyncpg for async FastAPI routes; simpler sync version with psycopg2 is fine to start
Caching	Redis	Cache schema introspection + repeated question -> SQL mappings
LLM	OpenAI gpt-4o-mini	Same model as Project 1, temperature=0 for deterministic SQL
Containerization	Docker + docker-compose	Postgres + Redis + API, same pattern as Project 1

WATCH OUT

Reuse the lazy-initialization pattern from DocuMind's ingestion.py (`get_llm()`, `get_db_pool()` as module-level singletons created on first call, not at import time). That bug cost real debugging time in Project 1 — no reason to hit it twice.

4. Folder Structure

```
nl-to-sql/
■■■ app/
■   ■■■ main.py                # FastAPI entrypoint
■   ■■■ config.py               # env vars, settings
■   ■■■ routes/
■   ■   ■■■ query.py            # POST /query
■   ■   ■■■ schema.py           # GET /schema (debug/inspection endpoint)
■   ■■■ core/
■   ■   ■■■ introspection.py    # reads Postgres schema, caches in Redis
■   ■   ■■■ sql_chain.py        # prompt assembly + LLM SQL generation
■   ■   ■■■ executor.py         # runs SQL, catches errors
■   ■   ■■■ correction.py       # the self-correction retry loop
■   ■   ■■■ safety.py           # read-only enforcement (SELECT-only)
■   ■   ■■■ prompts.py          # system prompt + few-shot examples
■   ■■■ db/
■   ■   ■■■ postgres.py         # asyncpg connection pool
■   ■■■ schemas/
■       ■■■ schemas.py          # Pydantic request/response models
■■■ tests/
■   ■■■ test_safety.py          # confirms DROP/DELETE/UPDATE are rejected
■   ■■■ test_correction.py      # confirms retry loop fixes bad SQL
■   ■■■ test_introspection.py
■■■ docker-compose.yml           # Postgres + Redis + API
■■■ Dockerfile
■■■ requirements.txt
■■■ .env.example
■■■ README.md
```

5. The Self-Correction Loop

This is the single most important piece of the project, and the thing that separates it from a toy demo. LLM-generated SQL is wrong often — wrong column name, bad join, invalid syntax, referencing a table that doesn't exist. A production-minded system doesn't just fail; it treats the database error as new information and tries again.

The loop, step by step

- Generate SQL from the question + schema (attempt 1)
- Execute it against Postgres inside a try/except
- If it succeeds → return results immediately, done
- If it fails → capture the *exact* database error message (not just "it failed")
- Feed the original question, the failed SQL, and the error message back to the LLM, explicitly asking it to fix the query
- Retry execution with the corrected SQL
- Cap retries at 3 attempts — after that, fail gracefully and tell the user the question couldn't be answered, rather than looping forever or burning API cost

```
# app/core/correction.py
from app.core.sql_chain import generate_sql, fix_sql
from app.core.executor import execute_query
from app.core.safety import is_read_only

MAX_ATTEMPTS = 3

async def answer_question(question: str, schema_context: str):
    sql = await generate_sql(question, schema_context)
    last_error = None

    for attempt in range(1, MAX_ATTEMPTS + 1):
        if not is_read_only(sql):
            return {"error": "Generated query was not read-only — rejected.", "sql": sql}

        try:
            rows = await execute_query(sql)
            return {"sql": sql, "rows": rows, "attempts": attempt}
        except Exception as e:
            last_error = str(e)
            if attempt == MAX_ATTEMPTS:
                break
            # Feed the actual DB error back to the LLM and ask it to fix the query
            sql = await fix_sql(question, sql, last_error, schema_context)

    return {
        "error": f"Could not produce a working query after {MAX_ATTEMPTS} attempts.",
        "last_sql_tried": sql,
        "last_error": last_error,
    }
```

KEY INSIGHT

The key detail interviewers probe on: the correction step doesn't just say "try again," it passes the **actual database error text** back into the next prompt. "column 'revenue' does not exist, did you mean 'total_revenue'?" is enormously more useful to the LLM than a generic retry — this is what makes the loop actually converge instead of repeating the same mistake three times.

app/core/sql_chain.py — generation & fixing

```
from app.core.prompts import SQL_GENERATION_PROMPT, SQL_FIX_PROMPT
from app.config import settings
from langchain_openai import ChatOpenAI

_llm = None

def get_llm():
    global _llm
    if _llm is None:
        _llm = ChatOpenAI(model=settings.chat_model, temperature=0)
    return _llm

async def generate_sql(question: str, schema_context: str) -> str:
    prompt = SQL_GENERATION_PROMPT.format(schema=schema_context, question=question)
    response = get_llm().invoke(prompt)
    return _extract_sql(response.content)

async def fix_sql(question: str, failed_sql: str, error: str, schema_context: str) -> str:
    prompt = SQL_FIX_PROMPT.format(
        schema=schema_context, question=question,
        failed_sql=failed_sql, error=error,
    )
    response = get_llm().invoke(prompt)
    return _extract_sql(response.content)

def _extract_sql(text: str) -> str:
    # Strip markdown code fences if the model wraps its answer in ```sql ... ```
    text = text.strip()
    if text.startswith("```"):
        text = text.split("```")[1]
        text = text.replace("sql", "", 1).strip()
    return text.strip().rstrip(";")
```

6. Safety Enforcement — Read-Only Guarantee

Prompting the model to "only write SELECT statements" is not a safety mechanism — it's a suggestion the model can ignore, especially after a retry where it gets creative trying to fix an error. The enforcement has to happen in code, before anything touches the real connection.

```
# app/core/safety.py
import re

FORBIDDEN = re.compile(
    r"\b(DROP|DELETE|UPDATE|INSERT|ALTER|TRUNCATE|GRANT|REVOKE|CREATE)\b",
    re.IGNORECASE,
)

def is_read_only(sql: str) -> bool:
    stripped = sql.strip().rstrip(";")
    if not stripped.upper().startswith("SELECT"):
        return False
    if FORBIDDEN.search(stripped):
        return False
    return True
```

SAFETY CRITICAL

Belt-and-suspenders: even with this check, run the query using a dedicated Postgres role that only has SELECT grants on the relevant tables. If the regex check has a gap, the database-level permission is the real backstop. This layered approach (app-level check + DB-level permission) is exactly the kind of defense-in-depth answer interviewers want to hear.

7. Schema Introspection

Before any SQL can be generated, the LLM needs to know what tables and columns actually exist. Postgres exposes this through information_schema — no need to hardcode it.

```
# app/core/introspection.py
async def get_schema_context(pool) -> str:
    query = """
    SELECT table_name, column_name, data_type
    FROM information_schema.columns
    WHERE table_schema = \'public\'
    ORDER BY table_name, ordinal_position;
    """
    async with pool.acquire() as conn:
        rows = await conn.fetch(query)

    tables = {}
    for r in rows:
        tables.setdefault(r["table_name"], []).append(f'{r["column_name"]} ({r["data_type"]})')

    lines = []
    for table, cols in tables.items():
        lines.append(f"Table {table}: " + ", ".join(cols))
    return "\n".join(lines)
```

Cache the result in Redis (schema rarely changes mid-session) instead of re-querying information_schema on every single question — this is the practical use for Redis in this project, alongside optionally caching repeated question→SQL mappings.

8. Prompt Design

```
SQL_GENERATION_PROMPT = """You are a PostgreSQL expert. Given the schema below and a
question, write a single SELECT query that answers it.

Rules:
- Only output the SQL query, nothing else.
- Only use SELECT. Never write INSERT, UPDATE, DELETE, DROP, or ALTER.
- Use exact table and column names from the schema below.
- For relative dates ("last month", "this year"), use CURRENT_DATE and
  interval arithmetic explicitly rather than guessing a literal date.
- If the question is ambiguous about sort order or limit, default to a
  reasonable interpretation (e.g. "top" = ORDER BY ... DESC LIMIT 5).

Schema:
{schema}

Question: {question}

SQL:"""

SQL_FIX_PROMPT = """The following SQL query failed when executed against PostgreSQL.

Schema:
{schema}

Original question: {question}
Query that failed:
{failed_sql}

Database error:
{error}

Write a corrected SELECT query that fixes this error and still answers the
original question. Only output the corrected SQL."""
```

Explicitly handling relative dates and ambiguous sort/limit language in the prompt (rather than hoping the model infers it consistently) is what separates a demo that works once from one that works reliably across varied phrasing.

9. API Endpoint Specification

Method	Path	Purpose
`POST`	`/query`	Ask a natural-language question, get SQL + results back
`GET`	`/schema`	Debug endpoint — view the introspected schema context

POST /query — request / response

```
// Request
{
  "question": "What were our top 5 products by revenue last quarter?"
}

// Response (success)
{
  "sql": "SELECT product_name, SUM(revenue) AS total FROM orders ...",
  "rows": [
    {"product_name": "Widget A", "total": 48210.50},
    {"product_name": "Widget B", "total": 39875.00}
  ],
  "attempts": 1
}

// Response (all retries exhausted)
{
  "error": "Could not produce a working query after 3 attempts.",
  "last_sql_tried": "SELECT ...",
  "last_error": "column \"prod_name\" does not exist"
}
```

10. Error Handling & Edge Cases

Case	Handling
Non-read-only SQL generated	Rejected before execution by safety.py — never reaches the DB
SQL fails to execute	Caught, error fed into self-correction loop, retried up to 3x
All retries exhausted	Return the last error + last SQL tried, not a silent failure
Question unrelated to schema	LLM should return a query that yields zero rows or a clear "cannot answer" — worth testing explicitly
Very large result set	Cap with a hard LIMIT server-side regardless of what the LLM generates, to protect response size and DB load
OpenAI API failure/timeout	Catch, return 502, log for debugging — same pattern as Project 1

11. Build Order Checklist

- [ ] Set up venv, install dependencies, create .env
- [ ] Spin up Postgres + Redis via docker-compose, seed with sample data (e.g. orders/products)
- [ ] Write and test get_schema_context() against the seeded DB — print the output, sanity check it
- [ ] Write safety.py, write tests confirming DROP/DELETE/UPDATE are rejected before anything else
- [ ] Write generate_sql() with a hardcoded simple question, verify output looks like valid SQL
- [ ] Write executor.py, manually run the generated SQL against Postgres
- [ ] Write the self-correction loop — deliberately break a query (wrong column name) and confirm the loop catches and fixes it
- [ ] Wire up FastAPI /query route
- [ ] Add Redis caching for schema context
- [ ] Test with a range of question phrasings — relative dates, "top N", ambiguous sorts
- [ ] Write tests, get docker-compose fully working from a clean clone
- [ ] Write README with setup instructions, sample questions, and example output
- [ ] Deploy (Render/Railway — same as Projects 1 and 3)

12. Interview Q&A

Q: What's actually hard about this project? Isn't it just calling an API?

A: The LLM call itself is the easy part. The hard part is everything that makes the output trustworthy: schema introspection so the model knows what exists, a self-correction loop that catches and fixes failed SQL using the real database error, and a read-only enforcement layer that's independent of the prompt. Without those three things it's a demo that breaks on the first typo in a column name.

Q: How does the self-correction loop actually work?

A: Generated SQL executes inside a try/except. On failure, the exact database error message (not a generic "it failed") is fed back into a follow-up prompt along with the original question and failed query, asking the model to fix it specifically. Capped at 3 attempts to bound cost and latency.

Q: How do you prevent a destructive query?

A: Two layers: an application-level regex/parse check that rejects anything not starting with SELECT or containing DROP/DELETE/UPDATE/INSERT/ALTER, and a database-level backstop — the connection used for these queries runs as a Postgres role with SELECT-only grants, so even a gap in the app-level check can't cause damage.

Q: How do you handle ambiguous language like "recent" or "top"?

A: Explicit instructions in the system prompt: relative dates resolve via CURRENT_DATE and interval arithmetic rather than a guessed literal date; "top N" defaults to ORDER BY ... DESC LIMIT N. Tested against a range of phrasings rather than just one example question.

Q: Why cache schema context in Redis?

A: Schema introspection queries information_schema, which rarely changes within a session. Re-running it on every single question adds latency and DB load for no benefit — caching it is the practical justification for Redis in this project, alongside optionally caching repeated question→SQL mappings.

Q: How would you evaluate accuracy in production?

A: A curated set of question/expected-SQL or question/expected-result pairs against a known schema, tracking both execution success rate (does it run at all) and correctness (does the result match). Also worth tracking how often the self-correction loop is needed — a high first-attempt failure rate signals the prompt or schema context needs work.

This reuses the LangChain + OpenAI setup and lazy-init pattern from Project 1 (DocuMind) almost entirely — the new surface area is schema introspection, the correction loop, and safety enforcement (Sections 5–7). That's where build time should actually go.